

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Zatrzaski, przerzutniki, rejestry

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 5

Studia dzienne

Informatyka

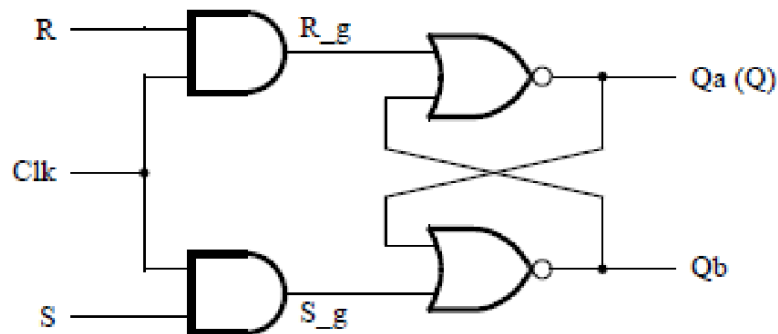
Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

Data: 31.03.2022

Zadanie 1

Utwórz dwa projekty zatrzasków RS z rys. 1.



Rys. 1. Układ zatrzasku RS utworzony z bramek logicznych

Jeden projekt **latch RS_gates** (rys. 2) używając prymitywów bramek języka **Verilog**, a drugi **latch RS_boolean** (rys. 3) za pomocą wyrażeń logicznych.

// bramkowy zatrzask RS

```
module RS_gates (Clk, R, S, Q);
    input Clk, R, S;
    output Q;
    wire R_g, S_g, Qa, Qb    /* synthesis keep */ ;
    and (R_g, R, Clk);
    and (S_g, S, Clk);
    nor (Qa, R_g, Qb);
    nor (Qb, S_g, Qa);
    assign Q = Qa;
endmodule
```

Rys. 2. Opis zatrzasku RS, poprzez wykorzystanie instancji elementów logicznych

// bramkowy zatrzask RS

```
module RS_boolean (Clk, R, S, Q);
    input Clk, R, S;
    output Q;
    wire R_g, S_g, Qa, Qb /* synthesis keep */ ;
    assign R_g = R & Clk;
    assign S_g = S & Clk;
    assign Qa = ~(R_g | Qb);
    assign Qb = ~(S_g | Qa);
    assign Q = Qa;
endmodule
```

Rys. 3. Definicja zatrzasku RS przy użyciu wyrażeń logicznych

Wykorzystaj dyrektywę **keep**, aby monitorować wewnętrzne węzły projektu w symulacji. Porównaj wyniki syntezy (**RTL Viewer**, **Technology Map Viewer**) bez używania i z użyciem dyrektywy **keep**.

Wskazówka: Symulator nie daje komunikatu o błędzie z początkowymi wartościami CLK = 1 i R = 1. Sprawdź, co podaje symulator dla R = 1 i S = 1.

Zaimplementuj projekt na płycie z następującymi pinami:

Wyprowadzenia	Piny
clk	SW[2]
R	SW[1]
S	SW[0]
Q	LEDR[0]

Kod w Verilog

Moduł zatrzasku RS z użyciem prymitywów bramek bez atrybutu „synthesis keep”.

```
module RS_gates(  
    input Clk, R, S,  
    output Q);  
  
    wire R_g, S_g, Qa, Qb;  
    and(R_g, R, Clk);  
    and(S_g, S, Clk);  
    nor(Qa, R_g, Qb);  
    nor(Qb, S_g, Qa);  
    assign Q = Qa;  
  
endmodule
```

Moduł zatrzasku RS z użyciem prymitywów bramek z atrybutem „synthesis keep”.

```
module RS_gates(  
    input Clk, R, S,  
    output Q);  
  
    wire R_g, S_g, Qa, Qb /* synthesis keep */;  
    and(R_g, R, Clk);  
    and(S_g, S, Clk);  
    nor(Qa, R_g, Qb);  
    nor(Qb, S_g, Qa);  
    assign Q = Qa;  
  
endmodule
```

Moduł zatrzasku RS z użyciem wyrażeń logicznych bez atrybutu „synthesis keep”.

```
module RS_boolean(  
    input Clk, R, S,  
    output Q);  
  
    wire R_g, S_g, Qa, Qb;  
    assign R_g = R & Clk;  
    assign S_g = S & Clk;  
    assign Qa = ~(R_g | Qb);  
    assign Qb = ~(S_g | Qa);  
    assign Q = Qa;  
  
endmodule
```

Moduł zatrasku RS z użyciem wyrażeń logicznych z atrybutem „synthesis keep”.

```
module RS_boolean(
    input Clk, R, S,
    output Q);

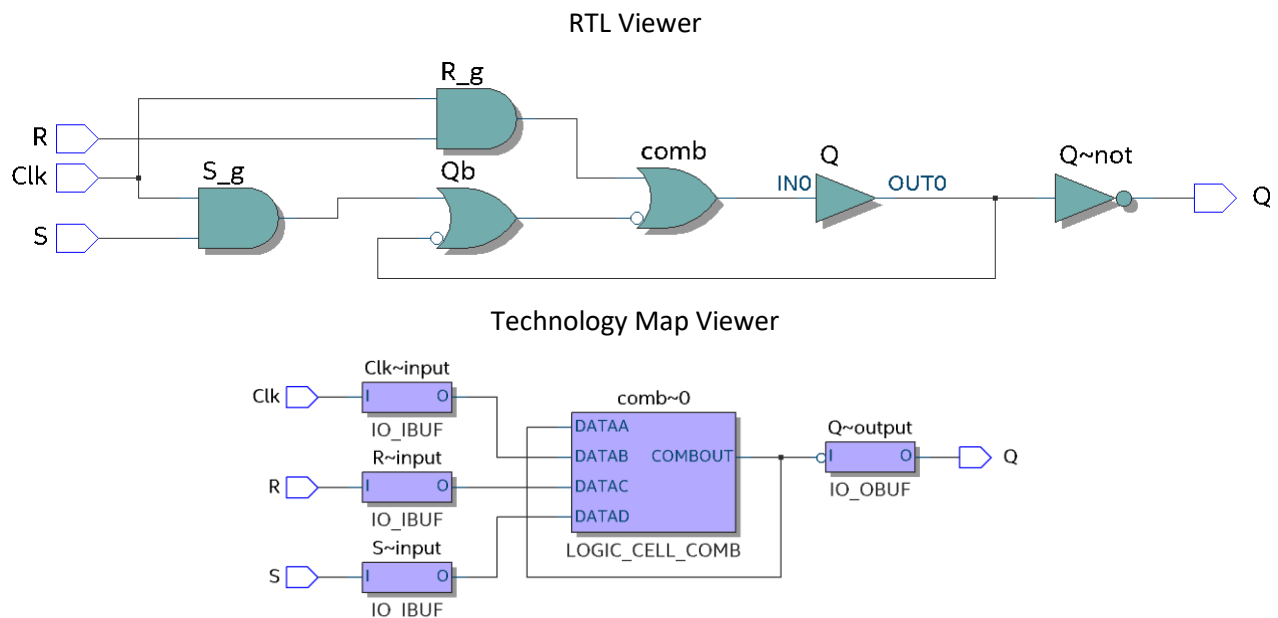
    wire R_g, S_g, Qa, Qb /* synthesis keep */;
    assign R_g = R & Clk;
    assign S_g = S & Clk;
    assign Qa = ~(R_g | Qb);
    assign Qb = ~(S_g | Qa);
    assign Q = Qa;

endmodule
```

Wyniki syntezy

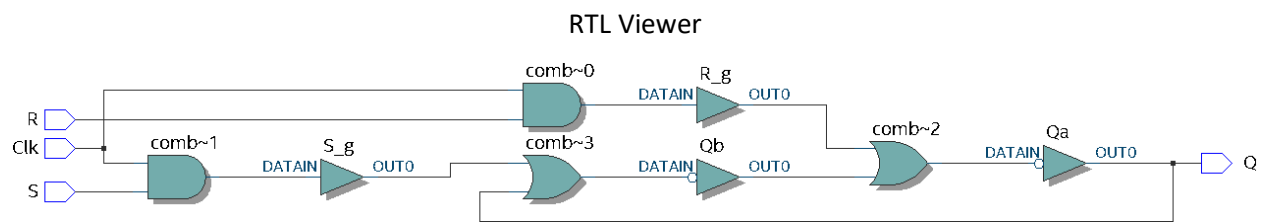
Moduł zatrasku RS z użyciem prymitywów bramek bez atrybutu „synthesis keep”.

Jeżeli nie użyjemy atrybutu /* synthesis keep */ w opisie modułu, to kompilator usunie węzły wewnętrzne, aby zminimalizować wynikowy układ. Przez to nie można wówczas monitorować węzłów wewnętrznych w symulacji.

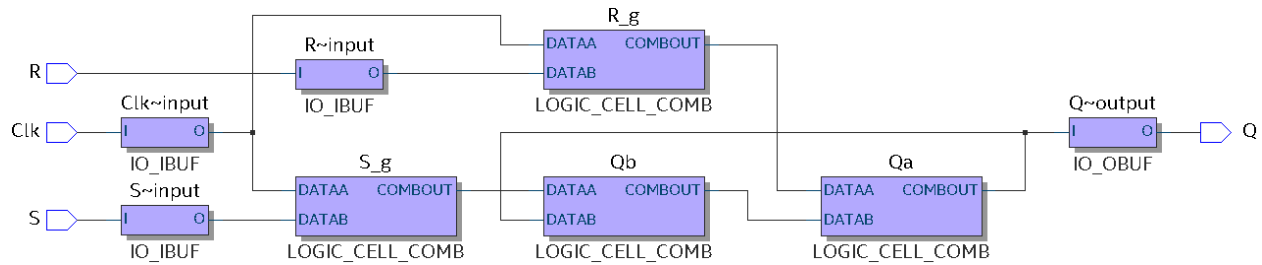


Moduł zatrasku RS z użyciem prymitywów bramek z atrybutem „synthesis keep”.

Jeżeli użyjemy atrybutu /* synthesis keep */ w opisie modułu, to kompilator pozostawi wskazane węzły wewnętrzne, co pozwoli je monitorować w symulacji.

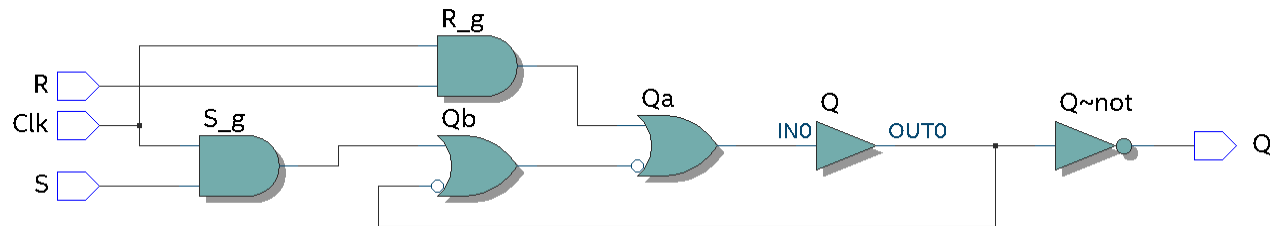


Technology Map Viewer

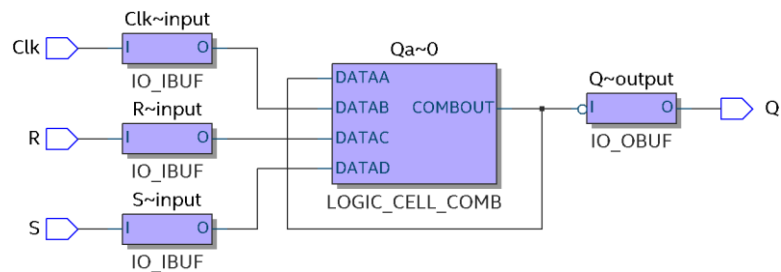


Moduł zatrzasku RS z użyciem wyrażeń logicznych bez atrybutu „synthesis keep”.

RTL Viewer

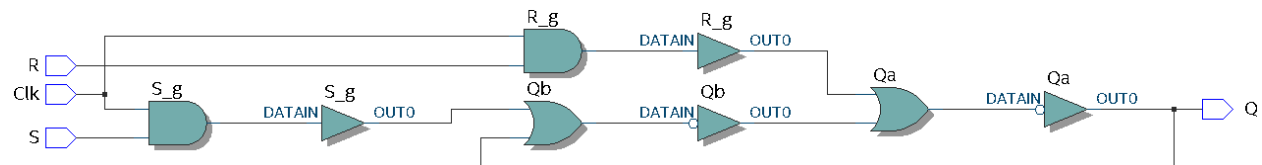


Technology Map Viewer

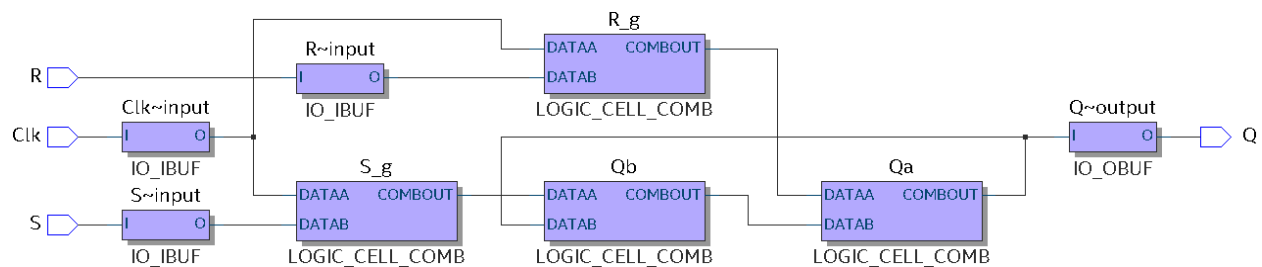


Moduł zatrzasku RS z użyciem wyrażeń logicznych z atrybutem „synthesis keep”.

RTL Viewer

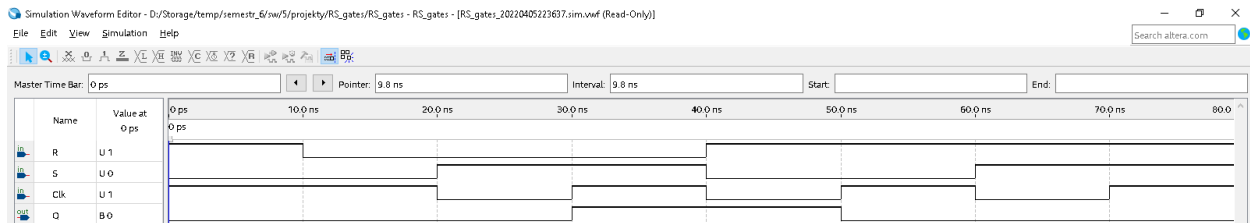


Technology Map Viewer

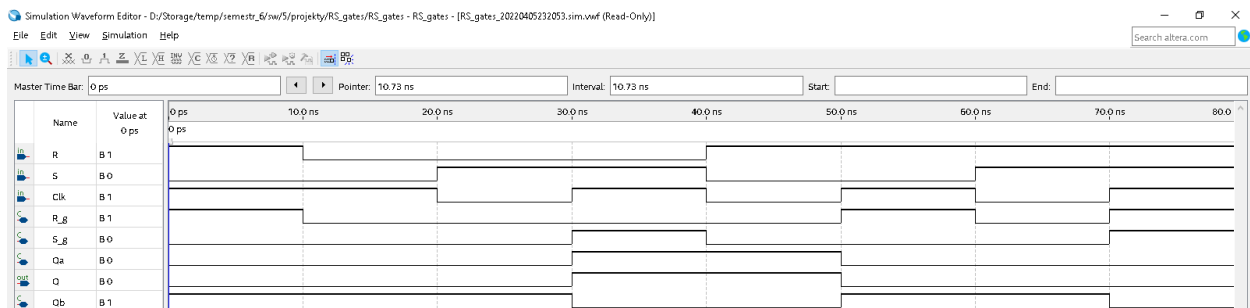


Wyniki symulacji funkcjonalnej

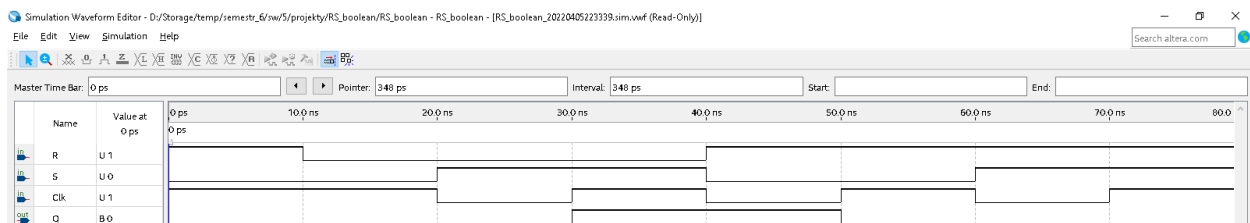
Moduł zatrasku RS z użyciem prymitywów bramek bez atrybutu „synthesis keep”.



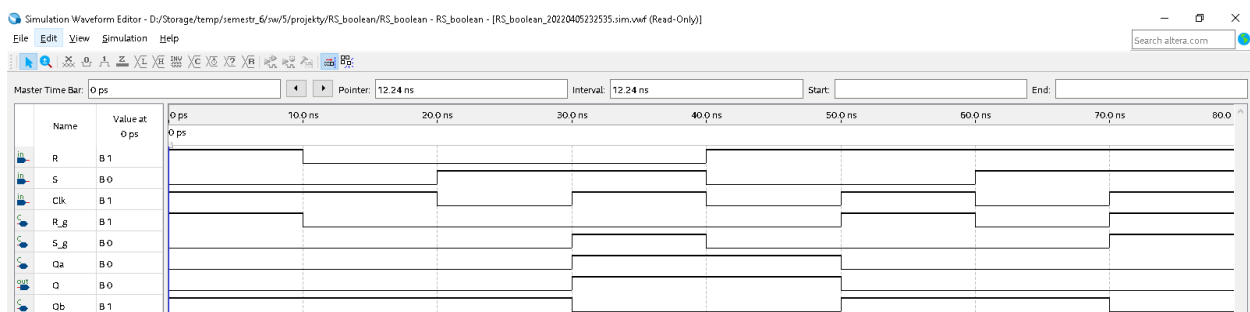
Moduł zatrasku RS z użyciem prymitywów bramek z atrybutem „synthesis keep”.



Moduł zatrasku RS z użyciem wyrażeń logicznych bez atrybutu „synthesis keep”.



Moduł zatrasku RS z użyciem wyrażeń logicznych z atrybutem „synthesis keep”.



Jeżeli kombinacja $R = 1$ i $S = 1$ wystąpi później niż na samym początku symulacji, to symulator ustawia wyjście $Q = 0$.

Jeżeli kombinacja $R = 1$ i $S = 1$ wystąpi na samym początku symulacji, to symulator wypisuje komunikat o błędzie.

Simulation Waveform Editor - D:\Storage\temp\semestr_6\sw\5\projekt\

File Edit View Simulation Help

Master Time Bar: 0 ps

	Name	Value at 0 ps
in	R	B 1
in	S	B 1
in	Clk	B 0
C	R_g	B U
C	S_g	B U
C	Qa	B U
out	Q	B 0
C	Qb	B U

Simulation Flow Progress

Error

```
# End time: 04:34:19 on Apr 06, 2022, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0

# vsim -novopt -c -t 1ps -L cyclonev_ver -L altera_ver -L altera_mf_ver -L 220model_ver -L sgate_ver -L
altera_insim_ver work.RS_gates_vlg_vec_tst
# Start time: 04:34:18 on Apr 06, 2022
# Loading work.RS_gates_vlg_vec_tst
# Loading work.RS_gates
# Loading cyclonev_ver.cyclonev_io_obuf
# Loading cyclonev_ver.cyclonev_io_ibuf
# Loading cyclonev_ver.cyclonev_lcell_comb
# after#25

# ** Error (suppressible): (vsim-3601) Iteration limit 5000 reached at time 0 ps.

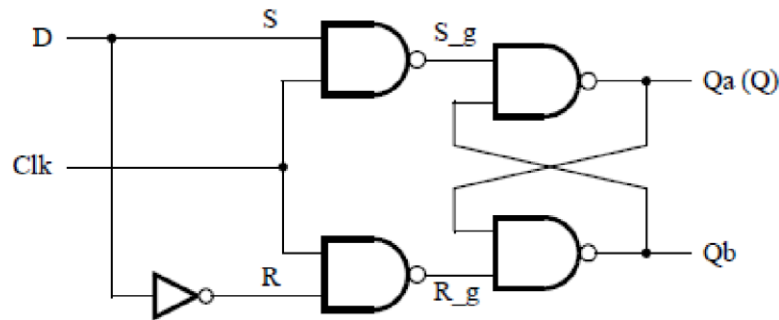
# End time: 04:34:19 on Apr 06, 2022, Elapsed time: 0:00:01
# Errors: 1, Warnings: 0

Error.
```

Zadanie 2

Utwórz projekt latch_D (rys. 4), używając dowolnego ze stylów opisu (używając prymitywów lub wyrażeń logicznych). Wykonaj wszystkie kroki z poprzedniego zadania dla tego projektu. Przydzielenie pinów:

Wyprowadzenia	Piny
D	SW[0]
clk	SW[1]
Q	LEDR[0]



Rys. 4. Schemat zatrasku D

Kod w Verilog

Moduł zatrasku D z użyciem wyrażeń logicznych bez atrybutu „synthesis keep”.

```
module latch_D(  
    input clk, D,  
    output Q);  
  
    wire S = D, R = ~D, S_g, R_g, Qa, Qb;  
    assign S_g = ~(S & clk);  
    assign R_g = ~(R & clk);  
    assign Qa = ~(S_g & Qb);  
    assign Qb = ~(R_g & Qa);  
    assign Q = Qa;  
  
endmodule
```

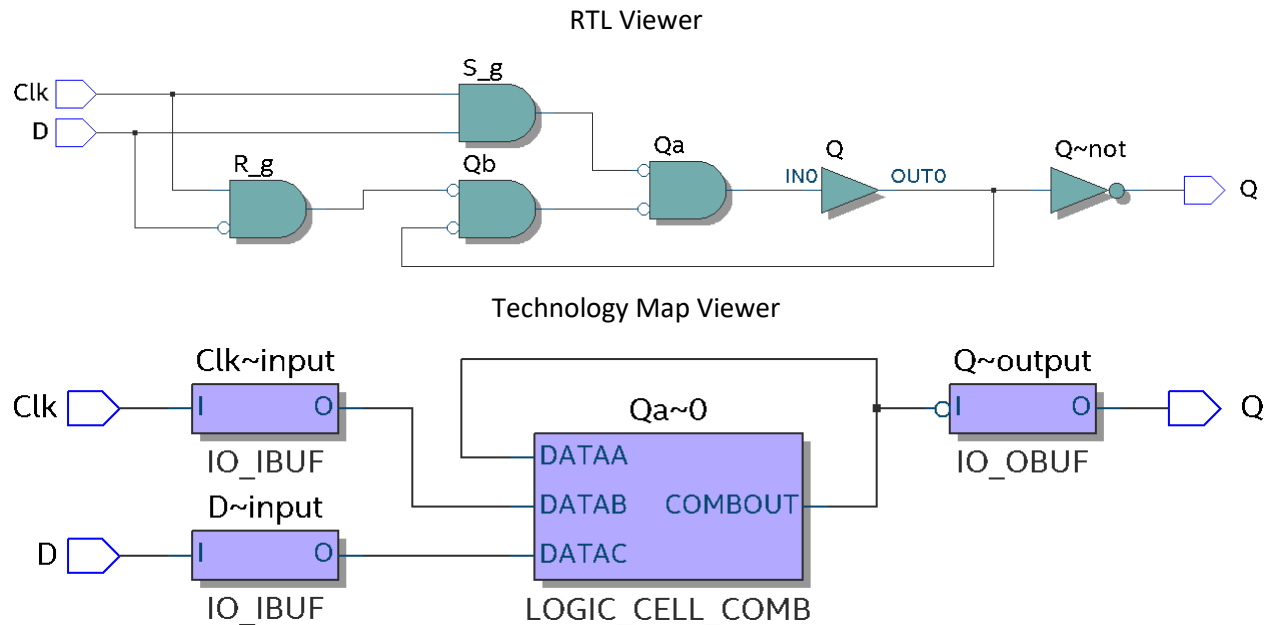
Moduł zatrasku D z użyciem wyrażeń logicznych z atrybutem „synthesis keep”.

```
module latch_D(  
    input clk, D,  
    output Q);  
  
    wire S = D, R = ~D, S_g, R_g, Qa, Qb /* synthesis keep */;  
    assign S_g = ~(S & clk);  
    assign R_g = ~(R & clk);  
    assign Qa = ~(S_g & Qb);  
    assign Qb = ~(R_g & Qa);  
    assign Q = Qa;  
  
endmodule
```


Wyniki syntezy

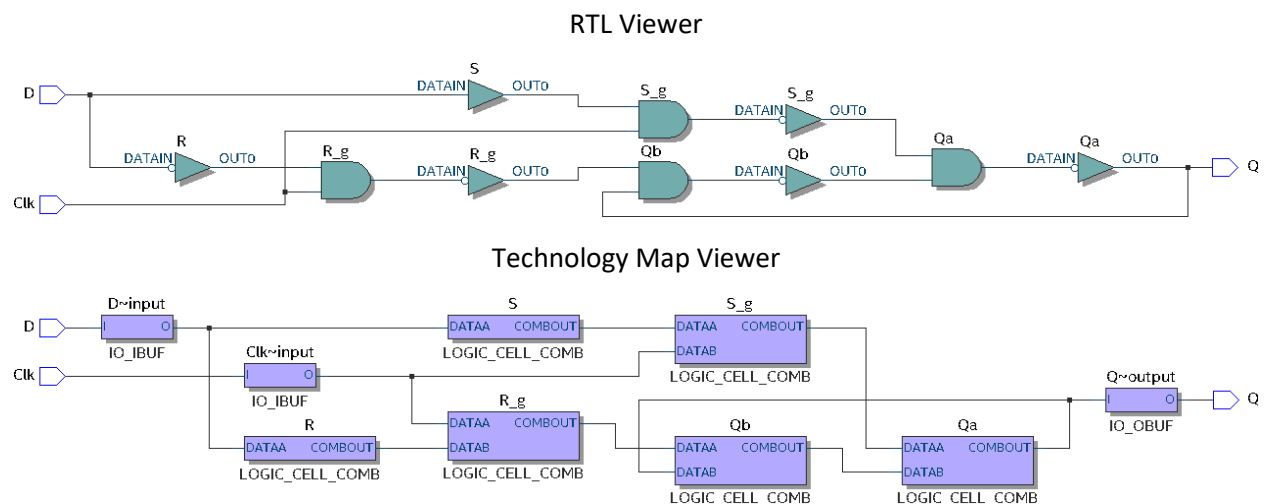
Moduł zatrasku D z użyciem wyrażeń logicznych bez atrybutu „synthesis keep”.

Bez użycia atrybutu `/* synthesis keep */` w opisie modułu kompilator usunie węzły wewnętrzne, aby zminimalizować wynikowy układ, przez co nie można ich monitorować w symulacji.



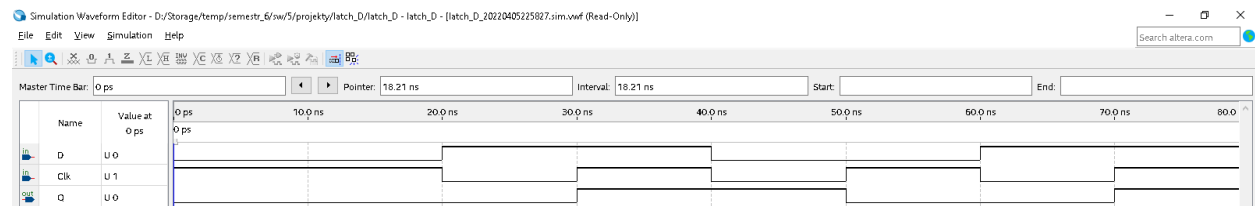
Moduł zatrasku D z użyciem wyrażeń logicznych z atrybutem „synthesis keep”.

Przy użyciu atrybutu `/* synthesis keep */` w opisie modułu kompilator pozostawi wskazane węzły wewnętrzne, dzięki czemu można je monitorować w symulacji.

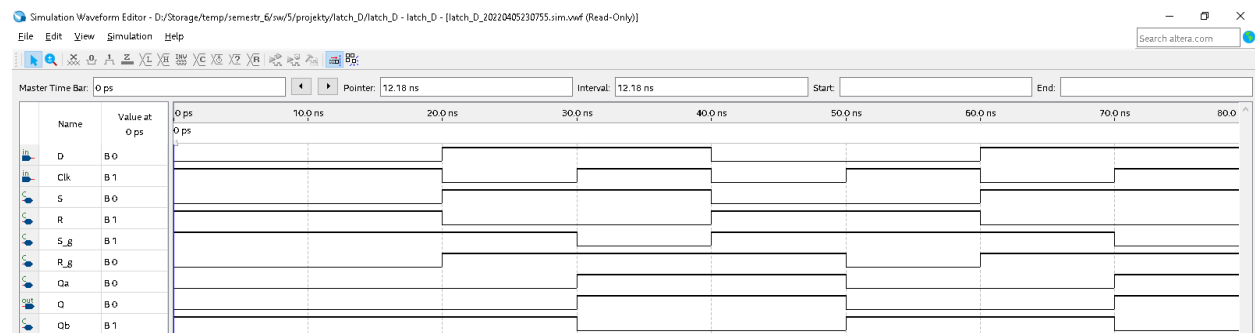


Wyniki symulacji funkcjonalnej

Moduł zatrasku D z użyciem wyrażeń logicznych bez atrybutu „synthesis keep”.



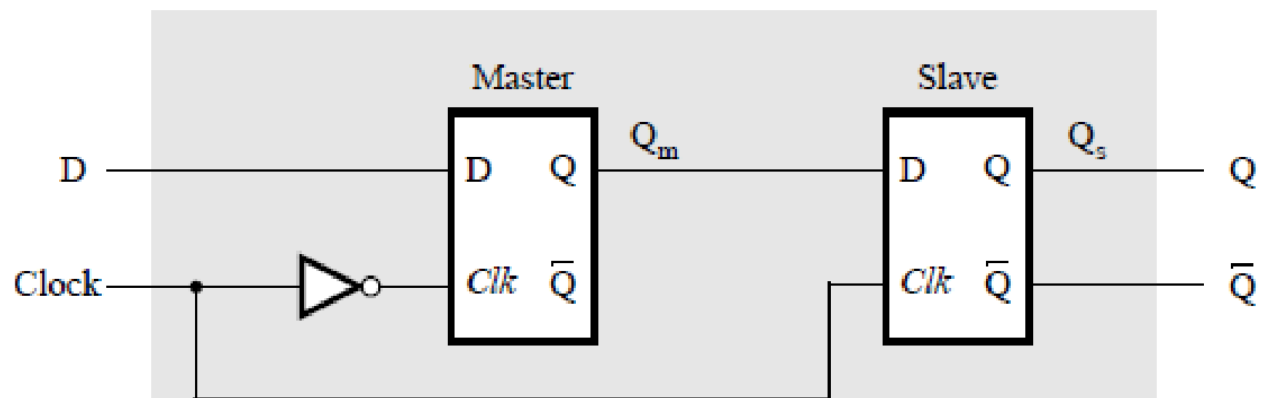
Moduł zatrasku D z użyciem wyrażeń logicznych z atrybutem „synthesis keep”.



Zadanie 3

Utwórz projekt master_slave_D (rysunek 5) przerzutnika D z dwóch zatrząsków. Użyj dyrektywy keep do monitorowania wewnętrznych węzłów. Porównaj wyniki syntezy (**RTL Viewer**, **Technology Map Viewer**) bez użycia i korzystając z dyrektywy keep. Uruchom symulację. Przy jakich wartościach początkowych symulator wyświetla komunikat o błędzie? Zaimplementować projekt na płycie z następującymi przypisaniami pinów:

Wyprowadzenia	Piny
D	SW[0]
clk	SW[1]
Q	LEDR[0]



Rys. 5. Schemat przerzutnika D **master-slave**

Kod w Verilog

Moduł przerzutnika D Master-Slave bez atrybutu „synthesis keep”.

```
module master_slave_D(  
    input D, clock,  
    output Q, nQ);  
  
    wire Qm, Qs;  
    latch_D master(~clock, D, Qm);  
    latch_D slave(clock, Qm, Qs);  
    assign Q = Qs;  
    assign nQ = ~Qs;  
  
endmodule
```

Moduł przerzutnika D Master-Slave z atrybutem „synthesis keep”.

```
module master_slave_D(  
    input D, clock,  
    output Q, nQ);  
  
    wire Qm, Qs /* synthesis keep */;  
    latch_D master(~clock, D, Qm);  
    latch_D slave(clock, Qm, Qs);  
    assign Q = Qs;  
    assign nQ = ~Qs;  
  
endmodule
```

Moduł zatrasku D z użyciem wyrażeń logicznych.

```
module latch_D(
    input clk, D,
    output Q);

    wire S = D, R = ~D, S_g, R_g, Qa, Qb;
    assign S_g = ~(S & clk);
    assign R_g = ~(R & clk);
    assign Qa = ~(S_g & Qb);
    assign Qb = ~(R_g & Qa);
    assign Q = Qa;

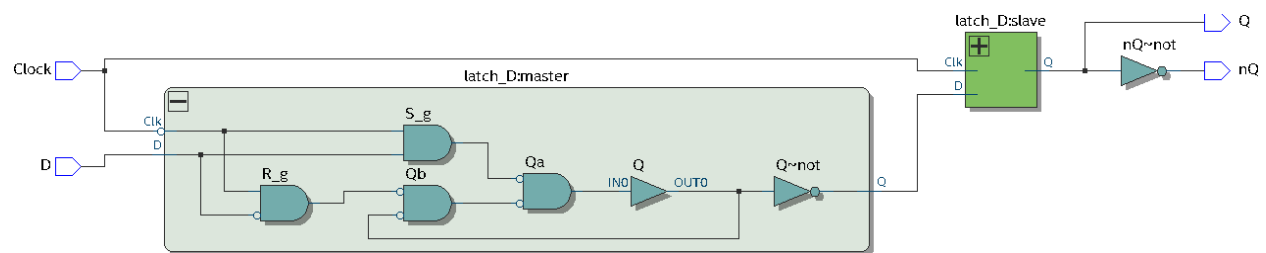
endmodule
```

Wyniki syntezy

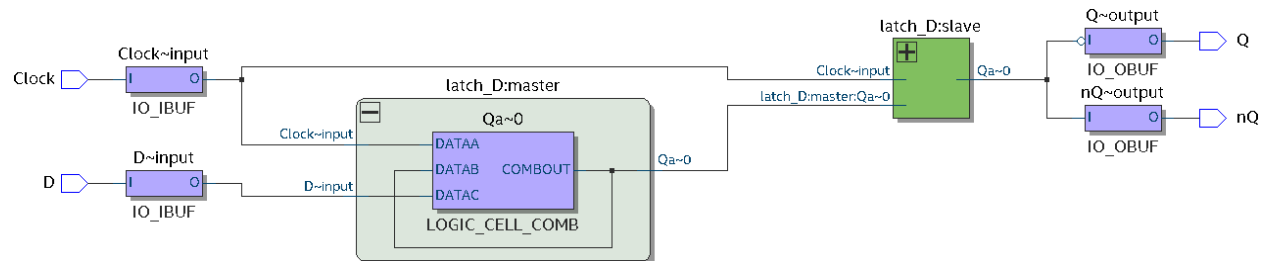
Moduł przerzutnika D Master-Slave bez atrybutu „synthesis keep”.

Bez użycia atrybutu `/* synthesis keep */` w opisie modułu kompilator usunie węzły wewnętrzne, aby zminimalizować wynikowy układ, przez co nie można ich monitorować w symulacji.

RTL Viewer



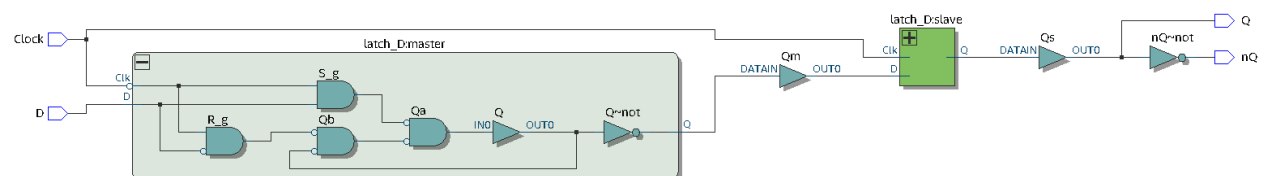
Technology Map Viewer



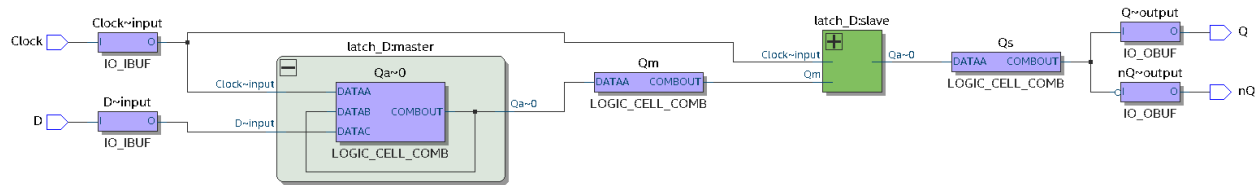
Moduł przerzutnika D Master-Slave z atrybutem „synthesis keep”.

Przy użyciu atrybutu `/* synthesis keep */` w opisie modułu kompilator pozostawi wskazane węzły wewnętrzne, dzięki czemu można je monitorować w symulacji.

RTL Viewer

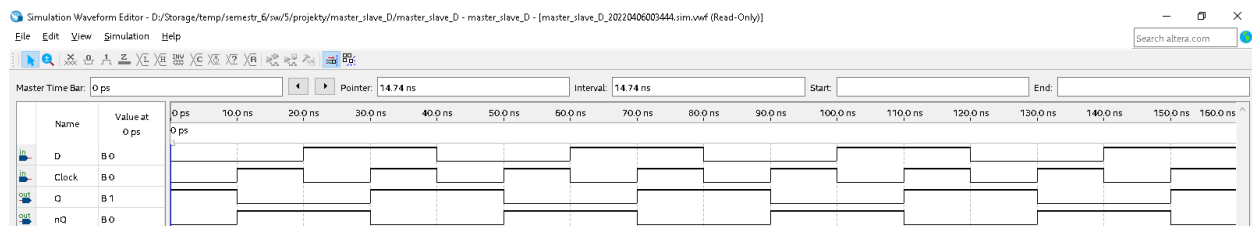


Technology Map Viewer

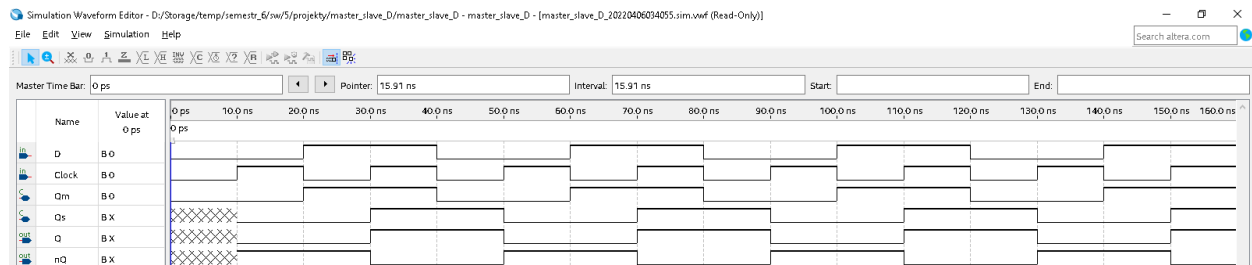


Wyniki symulacji funkcjonalnej

Moduł przerzutnika D Master-Slave bez atrybutu „synthesis keep”.

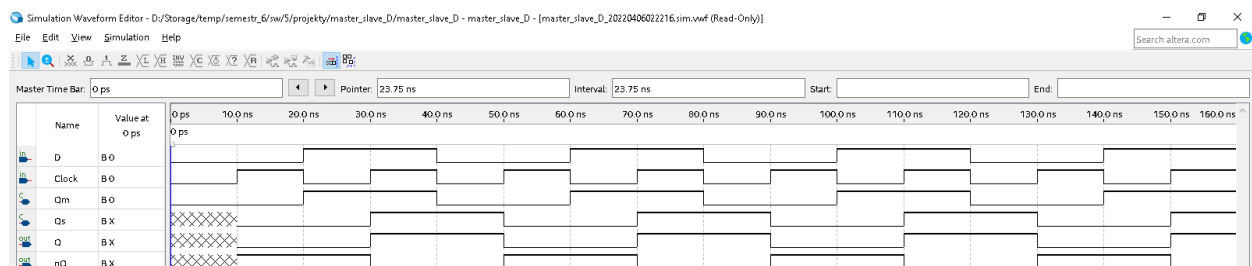


Moduł przerzutnika D Master-Slave z atrybutem „synthesis keep”.

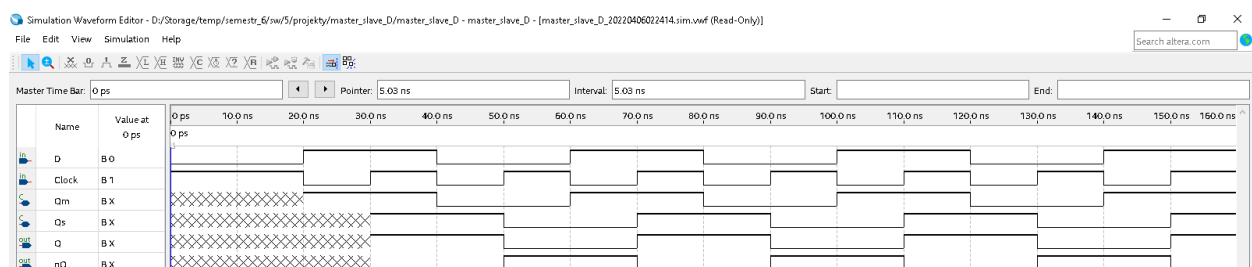


Przy żadnej kombinacji wartości początkowych symulator nie wyświetla komunikatu o błędzie.

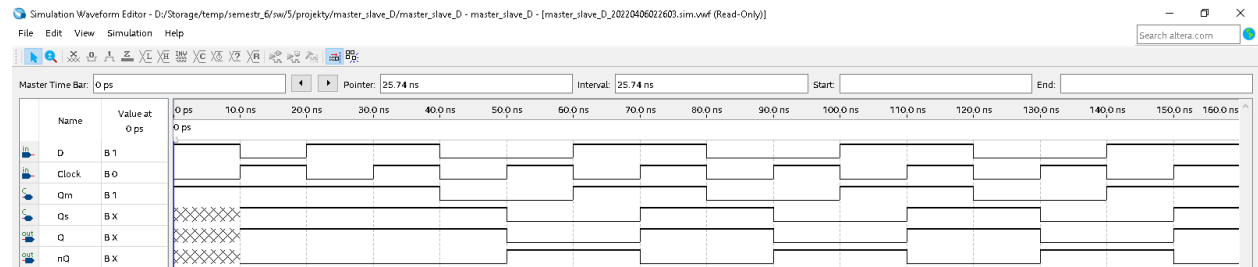
D = 0 i Clock = 0



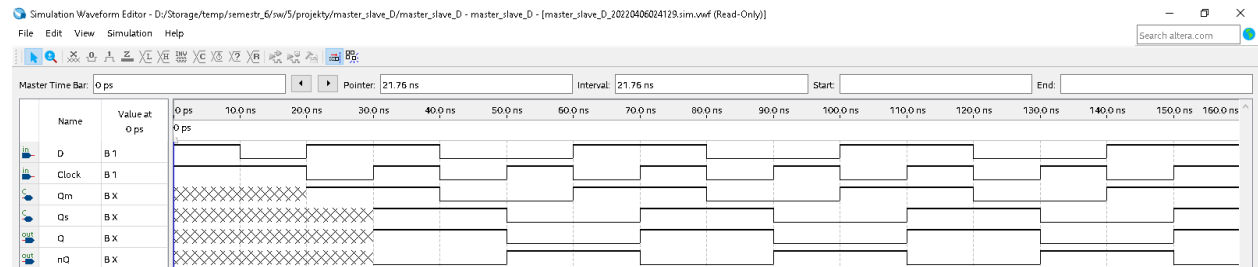
D = 0 i Clock = 1



D = 1 i Clock = 0

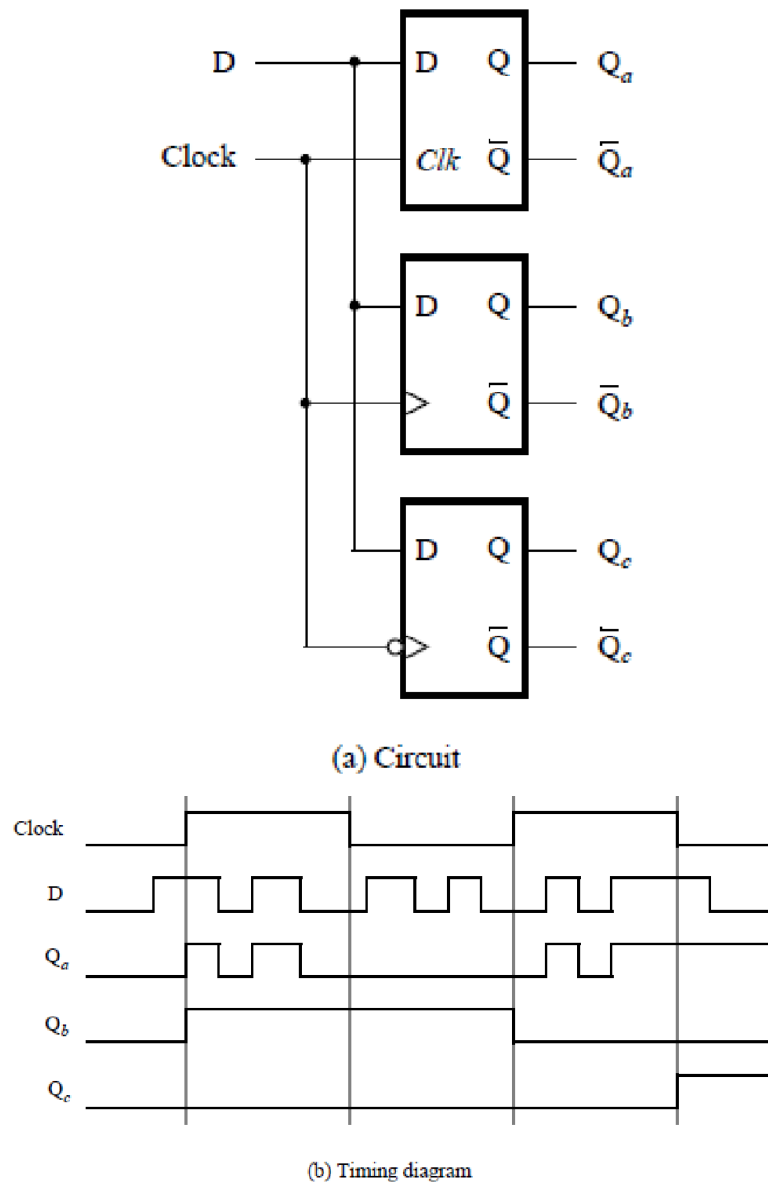


D = 1 i Clock = 1



Zadanie 4

Utwórz projekt latch_FDDP_FDDN, który implementuje układ z rysunku. 6, a. Sprawdź funkcjonowanie dla diagramu czasowego z rys. 6, b.



(a) Schemat

(b) diagram czasowy

Rys. 6. Schemat i diagramy czasowe dla projektu **latch_FDDP_FDDN**

Aby opisać zatrzask typu D (Rysunek 6), należy użyć przypisania proceduralnego języka Verilog (rysunek 7). Podobnie należy uczynić, aby opisać projekty przerzutników: **FDD_posedge** i **FDD_negedge**.

```

module D_latch (D, Clk, Q);
    input D, Clk;
    output reg Q;
    always @ (D, Clk)
        if (Clk)
            Q = D;
endmodule

```

Rys. 7. Proceduralny styl kodu **Verilog**, opisujący zatrzask typu D

Wdrożyć projekt na płycie z następującymi przypisaniami pinów:

Wyprowadzenia	Piny
D	SW[0]
clk	SW[1]
Qa	LEDR[0]
Qb	LEDR[1]
Qc	LEDR[2]

Kod w Verilog

Moduł najwyższego poziomu

```

module latch_FFDP_FFDN(
    input D, Clock,
    output Qa, nQa, Qb, nQb, Qc, nQc);

    D_latch latch(D, Clock, Qa, nQa);
    D_flip_flop_posedge ffp(D, Clock, Qb, nQb);
    D_flip_flop_negedge ffn(D, Clock, Qc, nQc);

endmodule

```

Zatrzask D

```

module D_latch(
    input D, Clk,
    output reg Q,
    output nQ);

    always @ (D, Clk)
        if (Clk) Q = D;
    assign nQ = ~Q;

endmodule

```


Przerzutnik D aktywowany zboczem narastającym

```
module D_flip_flop_posedge(
    input D, clk,
    output reg Q,
    output nQ);

    always @ (posedge clk)
        Q <= D;
    assign nQ = ~Q;

endmodule
```

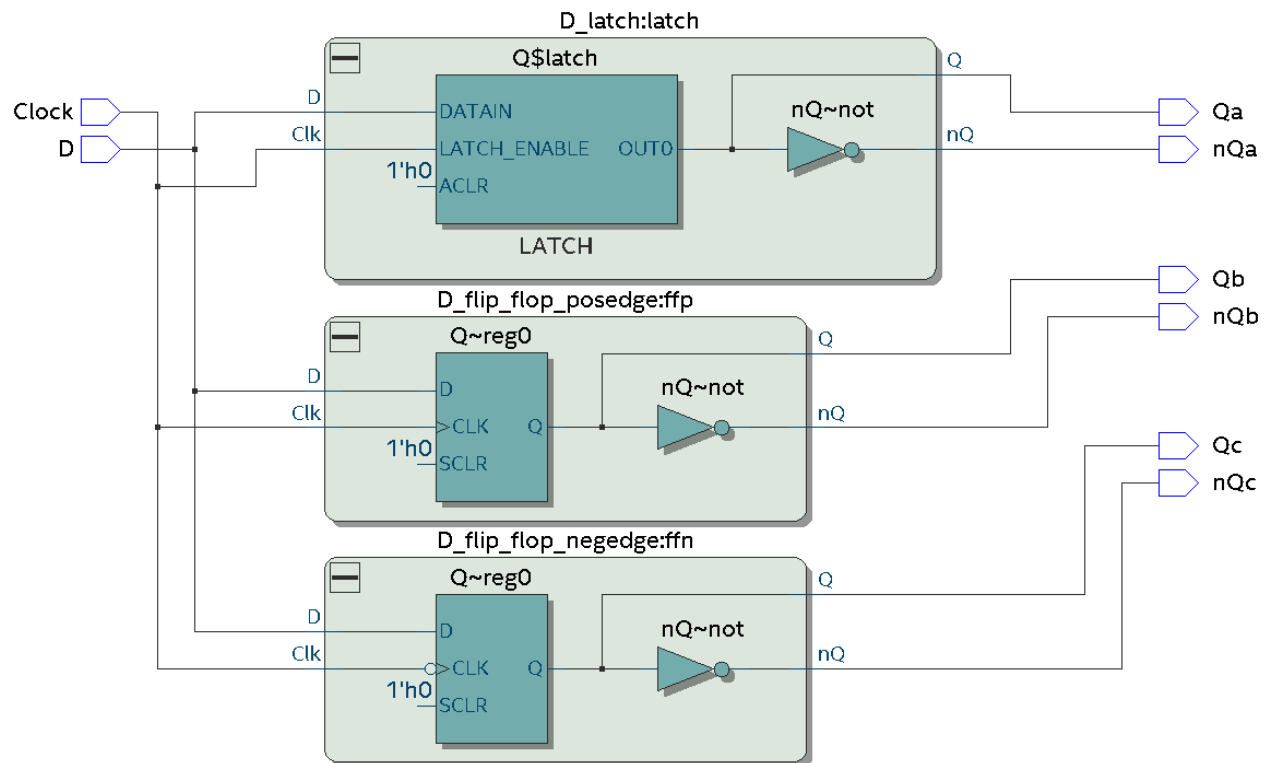
Przerzutnik D aktywowany zboczem opadającym

```
module D_flip_flop_negedge(
    input D, clk,
    output reg Q,
    output nQ);

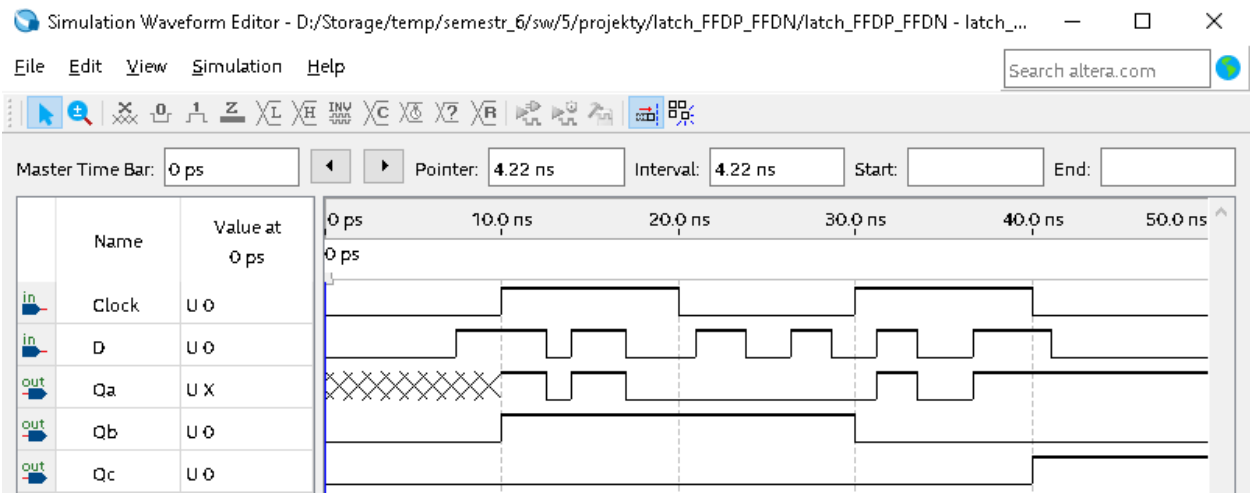
    always @ (negedge clk)
        Q <= D;
    assign nQ = ~Q;

endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



Zadanie 5

Utwórz projekt `adder_A_B_8_bits`, który dodaje dwie 8-bitowe liczby binarne A i B, wynik jest wyświetlany jako liczba szesnastkowa na ekranach HEX5 i HEX4. Liczby A i B są również wyświetlane w zapisie heksadecymalnym na wyświetlaczach 7-segmentowych: A na HEX3 i HEX2, B na HEX1 i HEX0. Wartości liczb A i B wprowadza się za pomocą przełączników SW7-0.

Wskazówka: Użyj multipleksowania podczas wprowadzania liczb A i B, najpierw wprowadź wartość liczby A i zapamiętaj ją w rejestrze, a następnie wprowadź wartość liczby B. Aby zapamiętać numer A, użyj rejestru `reg_N_bits_with_aretset` dla N bitów z asynchronicznym resetem. Zaimplementować projekt na płycie z następującymi przypisaniami pinów:

Wyprowadzenia	Piny
A, B	SW[7:0]
A	HEX3, HEX2
B	HEX1, HEX0
S = A + B	HEX5, HEX4
cout	LEDR[0]
areset	KEY0
clk	KEY1

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module adder_A_B_8_bits_on_board(  
    input [7:0] SW,  
    input [1:0] KEY,  
    output [6:0] HEX3, HEX2, HEX1, HEX0, HEX5, HEX4,  
    output [0:0] LEDR);  
  
    decoder_hex_16 A1(A[7:4], HEX3);  
    decoder_hex_16 A0(A[3:0], HEX2);  
    decoder_hex_16 B1(SW[7:4], HEX1);  
    decoder_hex_16 B0(SW[3:0], HEX0);  
    assign LEDR[0] = cout;  
    decoder_hex_16 S1(S[7:4], HEX5);  
    decoder_hex_16 S0(S[3:0], HEX4);  
  
    wire [7:0] A, S; // A - zapisany w rejestrze składnik sumy  
    wire cout;  
    // jeżeli KEY[1] = 1 (puszczony) -> 0 (wciśnięty), to ~KEY[1] = 0 -> 1, czyli wpisujemy A z SW do rejestru reg_A  
    adder_A_B_8_bits adder(SW, ~KEY[0], ~KEY[1], A, cout, S);  
  
endmodule
```

Sumator dwóch 8-bitowych liczb binarnych

```
module adder_A_B_8_bits(  
    input [7:0] A_B,  
    input reg_reset, reg_clock,  
    output [7:0] reg_value, // zapisany w rejestrze składnik sumy  
    output cout,  
    output [7:0] S);  
  
    // jeżeli reg_clock = 0 -> 1 (zbocze narastające), to wpisujemy A (aktualną wartość A_B) do rejestru reg_A  
    reg_N_bits_with_aretset reg_A(reg_reset, reg_clock, A_B, reg_value);  
    // sumujemy A zapisane w rejestrze reg_A z A_B  
    assign {cout, S} = reg_value + A_B;  
  
endmodule
```

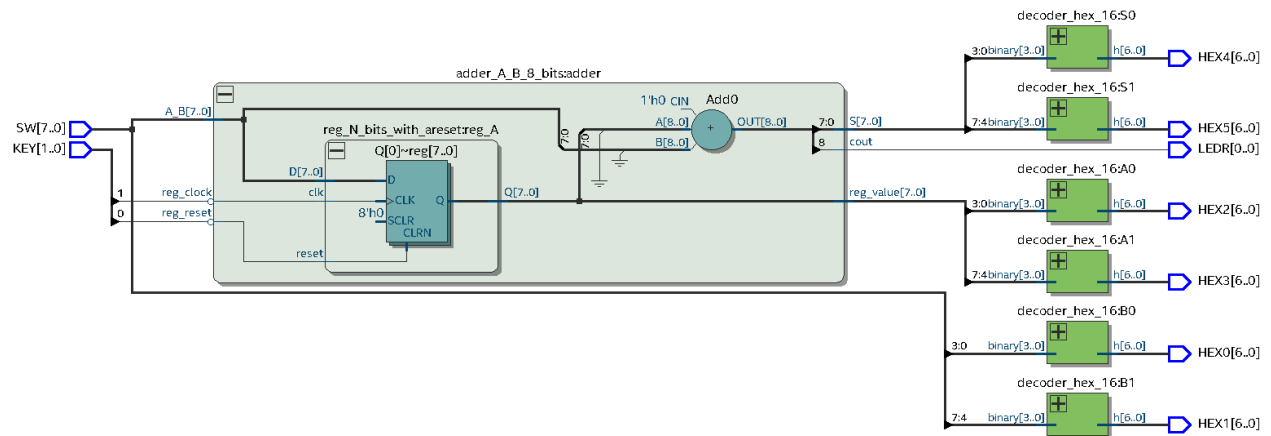
Rejestr 8-bitowy

```
module reg_N_bits_with_areset
#(parameter N = 8)(
    input reset, clk,
    input [(N-1):0] D,
    output reg [(N-1):0] Q);

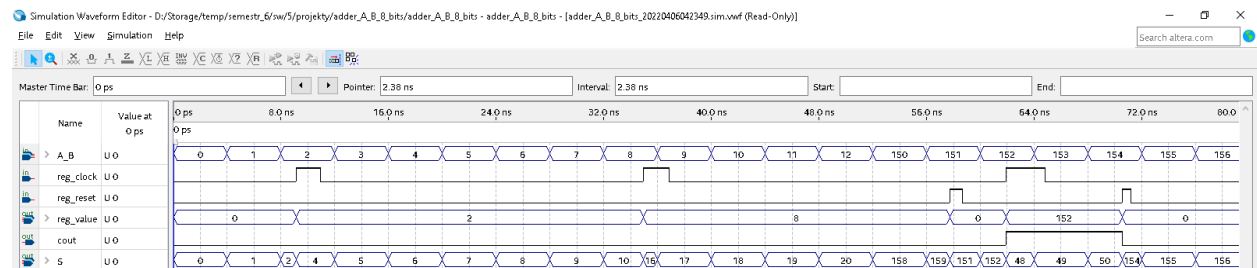
    always @ (posedge clk, posedge reset)
        if (reset == 1'b1) Q <= {N{1'b0}};
        // w momencie odebrania zbocza narastającego clk rejestr zapisuje D i zaczyna podawać je na Q
        else Q <= D;

endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



Wnioski

Wszystkie zadania zostały zrealizowane.